MODUL PRAKTIKUM 4

Streaming Processing + Real-Time Dashboard

(Industrial Style – Spark Structured Streaming + Streamlit)

Programming Language	: **Python**
Development Environment	: ***Linux Server Environment***
Code Editor	: **VS Code** (*Remote* ***WSL***)
Command Line Interface	: **Bash**
Distributed Data Processing Engine	: **Apache Spark (PySpark)**
Data Storage	: **Parquet Data Lake**
Real-Time Visualization	: **Streamlit**
Durasi	: **150 menit**

---

1. TUJUAN PRAKTIKUM

Mahasiswa mampu:

1. Memahami konsep stream processing dalam sistem Big Data.
2. Mensimulasikan event streaming menggunakan file incremental.
3. Mengimplementasikan Spark Structured Streaming untuk memproses data real-time.
4. Menyimpan hasil streaming ke Serving Layer (Parquet Data Lake).
5. Membangun dashboard analitik real-time menggunakan Streamlit.
6. Mengintegrasikan pipeline Streaming → Analytics → Visualization.

2. KONTEKS INDUSTRI

Banyak perusahaan teknologi memproses data secara real-time untuk pengambilan keputusan cepat.

Contoh implementasi:

Industri	Contoh
E-Commerce	*monitoring transaksi*
Fintech	*fraud detection*
Transportasi	*monitoring perjalanan*
IoT	*monitoring sensor*

Perusahaan seperti Uber dan Netflix menggunakan event *streaming architecture* untuk memproses jutaan event setiap detik.

Arsitektur umum streaming:

```
Event Producer
      ↓
Streaming Platform
      ↓
Stream Processing Engine
      ↓
Serving Layer
      ↓
Real-Time Dashboard
```

Dalam praktikum ini pipeline tersebut disimulasikan secara sederhana.

3. ARSITEKTUR PIPELINE

Arsitektur praktikum:

```
Transaction Generator
        |
        ▼
stream_data/
(JSON events)
        |
        ▼
Spark Structured Streaming
        |
        ▼
Parquet Data Lake
data/serving/stream
        |
        ▼
Real-Time Dashboard
(Streamlit)
```

Pipeline ini menunjukkan *end-to-end* streaming analytics system.

3.1 KONSEP YANG DIGUNAKAN

Event Streaming

Event streaming adalah proses pengiriman data secara kontinu saat event terjadi.

Contoh event:

- transaksi pembelian
- login user
- data sensor

File Incremental Streaming

Streaming disimulasikan dengan menambahkan file baru ke folder.
Spark membaca file tersebut sebagai data streaming.

Contoh:

```
transaction_1.json
transaction_2.json
transaction_3.json
```

Spark Structured Streaming

Spark Structured Streaming memungkinkan pemrosesan data streaming menggunakan API DataFrame.

Keunggulan:
- scalable
- fault tolerant
- integrasi dengan Spark SQL

Data Lake (Parquet)

Data hasil streaming disimpan dalam format Parquet.

Keunggulan:
- columnar storage
- ukuran file kecil
- cepat untuk analytics

Real-Time Dashboard

Dashboard digunakan untuk memvisualisasikan data streaming secara real-time.

Framework yang digunakan:

```
Streamlit
```

4. LANGKAH-LANGKAH PRAKTIKUM

LANGKAH 1

Struktur Project

Pastikan struktur project seperti berikut (*Tetap di File Project praktikum 1, 2, dan 3*):

```
BIGDATA-PROJECT
│
├── stream_data
│
├── data
│   └── serving
│       └── stream
│
├── scripts
│   ├── streaming_layer.py
│   └── transaction_generator.py
│
├── dashboard
│   └── dashboard_streamlit.py
│
├── logs
```

LANGKAH 2

Install Library

Aktifkan virtual environment.

```
source venv/bin/activate
```

Install dependency:

```
pip install streamlit pandas pyarrow
```

LANGKAH 3

Membuat Transaction Generator

Buat File:

`scripts/transaction_generator.py`

isi dengan coding berikut :

```
import json
import random
import time
import os
from datetime import datetime

output_folder = "stream_data"

products = ["Laptop","Mouse","Keyboard","Monitor","Headset","Webcam"]
cities = ["Jakarta","Bandung","Surabaya","Medan","Yogyakarta"]

counter = 1

while True:

    transaction = {
        "user_id": random.randint(100,200),
        "product": random.choice(products),
        "price": random.randint(50,2000),
        "city": random.choice(cities),
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }

    filename = f"transaction_{counter}.json"
    filepath = os.path.join(output_folder,filename)

    with open(filepath,"w") as f:
        json.dump(transaction,f)

    print("Generated:",transaction)

    counter += 1

    time.sleep(3)
```

Script ini akan menghasilkan event transaksi setiap 3 detik.

LANGKAH 4

Membuat Spark Streaming Script

Buat File :

`scripts/streaming_layer.py`

isi dengan coding berikut :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder \
    .appName("StreamingPipeline") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

schema = """
user_id INT,
product STRING,
price DOUBLE,
city STRING,
timestamp STRING
"""

stream_df = spark.readStream \
    .schema(schema) \
    .option("maxFilesPerTrigger",1) \
    .json("stream_data")

query = stream_df.writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path","data/serving/stream") \
    .option("checkpointLocation","logs/stream_checkpoint") \
    .trigger(processingTime="5 seconds") \
    .start()

query.awaitTermination()
```

LANGKAH 5

Membuat Real-Time Dashboard (*Industrial Style*)

Buat File:

`dashboard/dashboard_streamlit.py`

isi dengan coding berikut :

```
import streamlit as st
import pandas as pd
import os
import time
```

```
st.set_page_config(
    page_title="Real-Time E-Commerce Dashboard",
    layout="wide"
)

st.title("Real-Time E-Commerce Analytics Dashboard")

DATA_PATH = "data/serving/stream"
placeholder = st.empty()
def load_stream_data():

    if not os.path.exists(DATA_PATH):
        return pd.DataFrame()

    files = [f for f in os.listdir(DATA_PATH) if f.endswith(".parquet")]

    if len(files) == 0:
        return pd.DataFrame()

    df = pd.concat(
        [pd.read_parquet(os.path.join(DATA_PATH, f)) for f in files],
        ignore_index=True
    )
    return df

while True:
    with placeholder.container():
        try:
            df = load_stream_data()

            if df.empty:
                st.info("Waiting for streaming data...")
                time.sleep(5)
                continue

            # convert timestamp
            if "timestamp" in df.columns:
                df["timestamp"] = pd.to_datetime(df["timestamp"])

            # ========================
            # KPI METRICS
            # ========================

            st.subheader("Key Metrics")

            col1, col2, col3, col4 = st.columns(4)

            total_transactions = len(df)
            total_revenue = df["price"].sum()
            avg_transaction = df["price"].mean()
            unique_cities = df["city"].nunique()
```

```
col1.metric("Total Transactions", total_transactions)
col2.metric("Total Revenue", f"${int(total_revenue)}")
col3.metric("Avg Transaction", f"${int(avg_transaction)}")
col4.metric("Cities", unique_cities)

st.divider()

# ========================
# CHART SECTION
# ========================

colA, colB = st.columns(2)

# Revenue per City
with colA:

    st.subheader("Revenue by City")

    city_sales = (
        df.groupby("city")["price"]
        .sum()
        .sort_values(ascending=False)
    )

    st.bar_chart(city_sales)

# Top Products
with colB:

    st.subheader("Top Products")

    product_sales = (
        df.groupby("product")["price"]
        .sum()
        .sort_values(ascending=False)
    )

    st.bar_chart(product_sales)

st.divider()

# ========================
# REVENUE TREND
# ========================

st.subheader("Revenue Trend")

if "timestamp" in df.columns:

    revenue_trend = (
        df.set_index("timestamp")
        .resample("10s")["price"]
        .sum()
    )

    st.line_chart(revenue_trend)

st.divider()
```

```
            # ========================
            # LIVE TRANSACTIONS TABLE
            # ========================

            st.subheader("Live Transactions")

            st.dataframe(
                df.sort_values("timestamp", ascending=False).head(30),
                use_container_width=True
            )

        except Exception as e:

            st.error("Streaming data not ready")
            st.text(str(e))

    time.sleep(5)
```

Dashboard menampilkan:

- KPI metrics
- revenue per city
- top products
- revenue trend
- live transaction table

LANGKAH 6

Menjalankan Pipeline

Gunakan 3 terminal.

Terminal 1 — Spark Streaming

```
spark-submit scripts/streaming_layer.py
```

Terminal 2 — Transaction Generator

```
python3 scripts/transaction_generator.py
```

Terminal 3 — Dashboard

```
python -m streamlit run dashboard/dashboard_streamlit.py
```

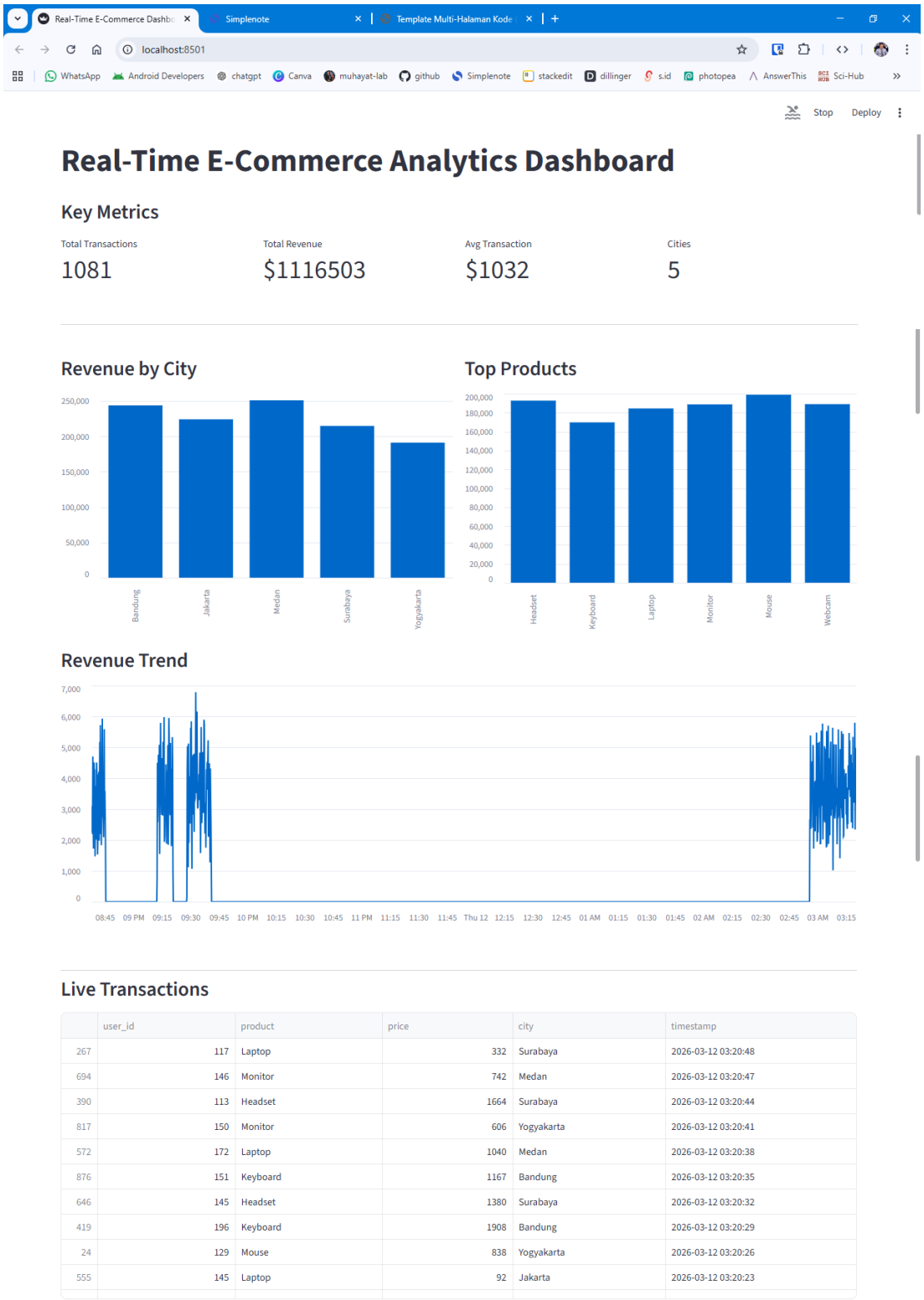
LANGKAH 7

Membuka Dashboard

Buka browser:

http://localhost:8501

Dashboard akan menampilkan analitik transaksi real-time.



	user_id	product	price	city	timestamp
267	117	Laptop	332	Surabaya	2026-03-12 03:20:48
694	146	Monitor	742	Medan	2026-03-12 03:20:47
390	113	Headset	1664	Surabaya	2026-03-12 03:20:44
817	150	Monitor	606	Yogyakarta	2026-03-12 03:20:41
572	172	Laptop	1040	Medan	2026-03-12 03:20:38
876	151	Keyboard	1167	Bandung	2026-03-12 03:20:35
646	145	Headset	1380	Surabaya	2026-03-12 03:20:32
419	196	Keyboard	1908	Bandung	2026-03-12 03:20:29
24	129	Mouse	838	Yogyakarta	2026-03-12 03:20:26
555	145	Laptop	92	Jakarta	2026-03-12 03:20:23

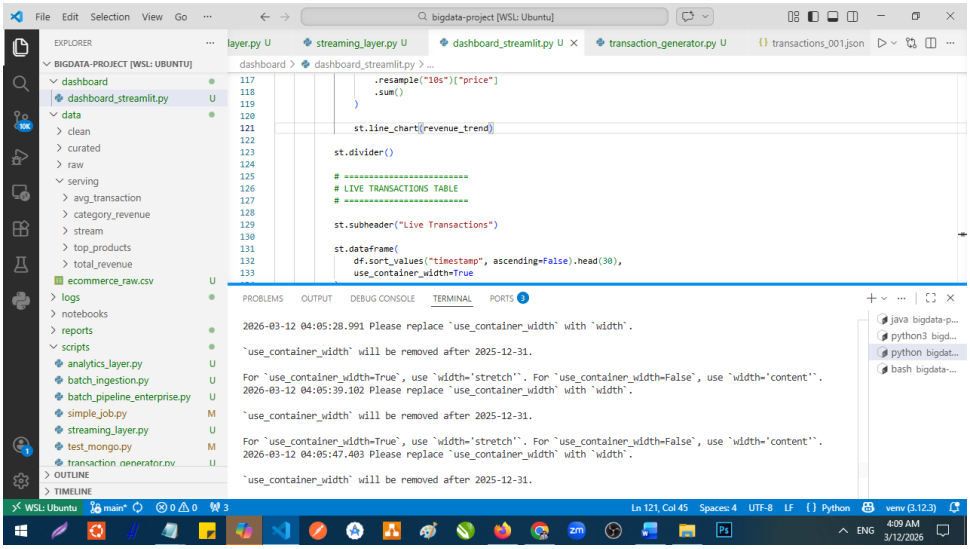
5. CHECKLIST VALIDASI PRAKTIKUM

No	Validasi	Status
1	Folder stream_data dibuat	✓
2	Generator menghasilkan transaksi	✓
3	Spark streaming berjalan	✓
4	File parquet muncul di serving/stream	✓
5	Dashboard menampilkan grafik	✓

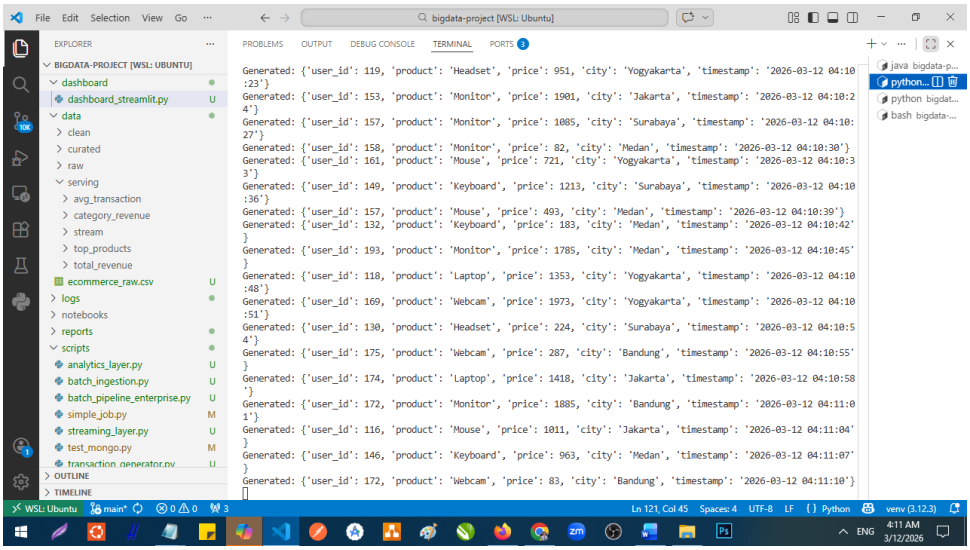
6. OUTPUT WAJIB YANG DIKUMPULKAN

Mahasiswa wajib mengumpulkan:

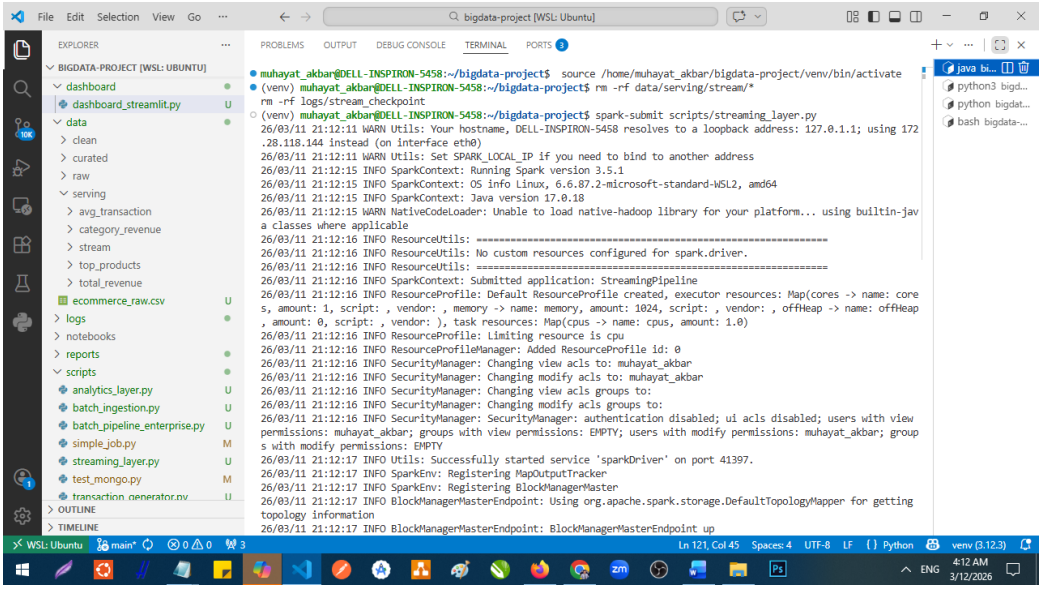
1. Screenshot struktur project



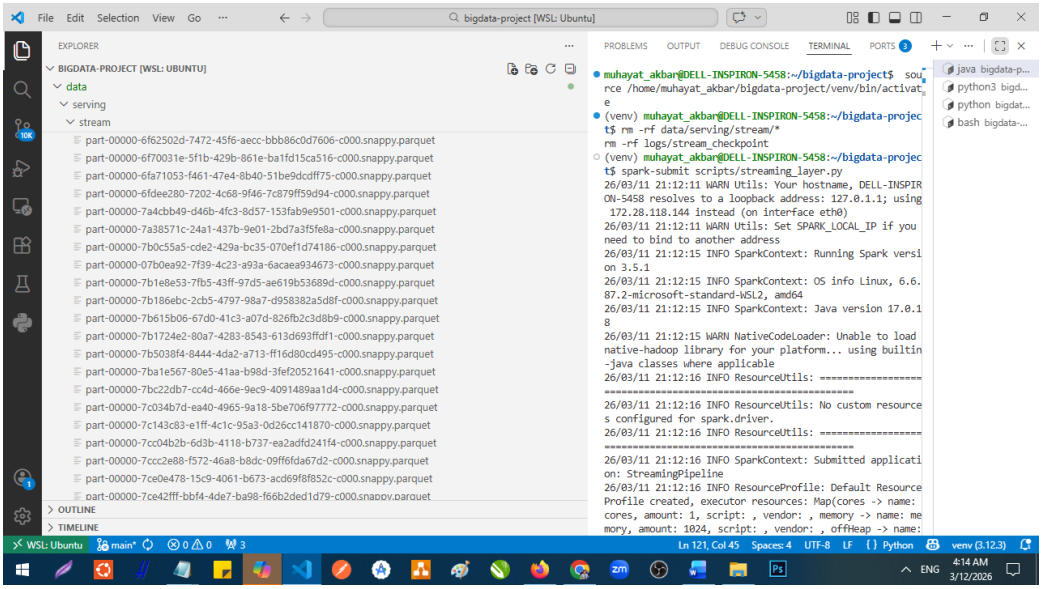
2. Screenshot generator transaksi berjalan



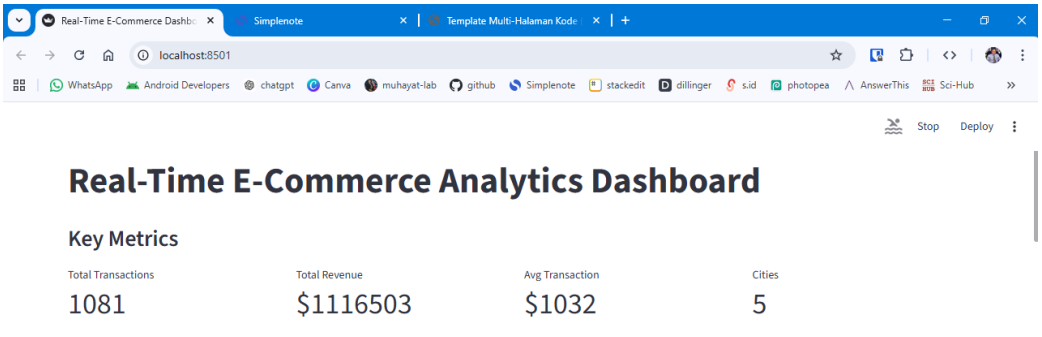
3. Screenshot Spark streaming berjalan



4. Screenshot folder data/serving/stream



5. Screenshot dashboard real-time



	user_id	product	price	city	timestamp
267	117	Laptop	332	Surabaya	2026-03-12 03:20:48
694	146	Monitor	742	Medan	2026-03-12 03:20:47
390	113	Headset	1664	Surabaya	2026-03-12 03:20:44
817	150	Monitor	606	Yogyakarta	2026-03-12 03:20:41
572	172	Laptop	1040	Medan	2026-03-12 03:20:38
876	151	Keyboard	1167	Bandung	2026-03-12 03:20:35
646	145	Headset	1380	Surabaya	2026-03-12 03:20:32
419	196	Keyboard	1908	Bandung	2026-03-12 03:20:29
24	129	Mouse	838	Yogyakarta	2026-03-12 03:20:26
555	145	Laptop	92	Jakarta	2026-03-12 03:20:23

File yang dikumpulkan:

```
streaming_layer.py
transaction_generator.py
dashboard_streamlit.py
```

7. RUBRIK PENILAIAN

Aspek	Skor 1	Skor 2	Skor 3	Skor 4
Setup Environment	gagal	sebagian	berhasil	optimal
Streaming Pipeline	tidak berjalan	sebagian	berjalan	stabil
Dashboard Analytics	tidak ada	basic	cukup	interaktif
Dokumentasi	tidak ada	minim	cukup	lengkap

Nilai akhir:

Nilai = **(Total Skor / 16) × 100**

8. INSIGHT PRAKTIKUM

Praktikum ini memperkenalkan konsep ***real-time analytics pipeline***.

Sistem Big Data modern biasanya terdiri dari beberapa lapisan:

```
Data Ingestion
      ↓
Stream Processing
      ↓
Serving Layer
      ↓
Real-Time Analytics
```

Pipeline yang dibuat pada praktikum ini merupakan mini data *platform real-time.* Konsep ini digunakan oleh banyak perusahaan teknologi besar untuk memproses data streaming dalam skala besar.

Catatan :

Jika semua telah berjalan tanpa error, dan screenshot tugas telah selesai di ambil, jangan lupa **hentikan (Stop) Pipeline pada masing-masing** terminal.

Terminal 1 — Spark Streaming

```
spark-submit scripts/streaming_layer.py
```

Terminal 2 — Transaction Generator

```
python3 scripts/transaction_generator.py
```

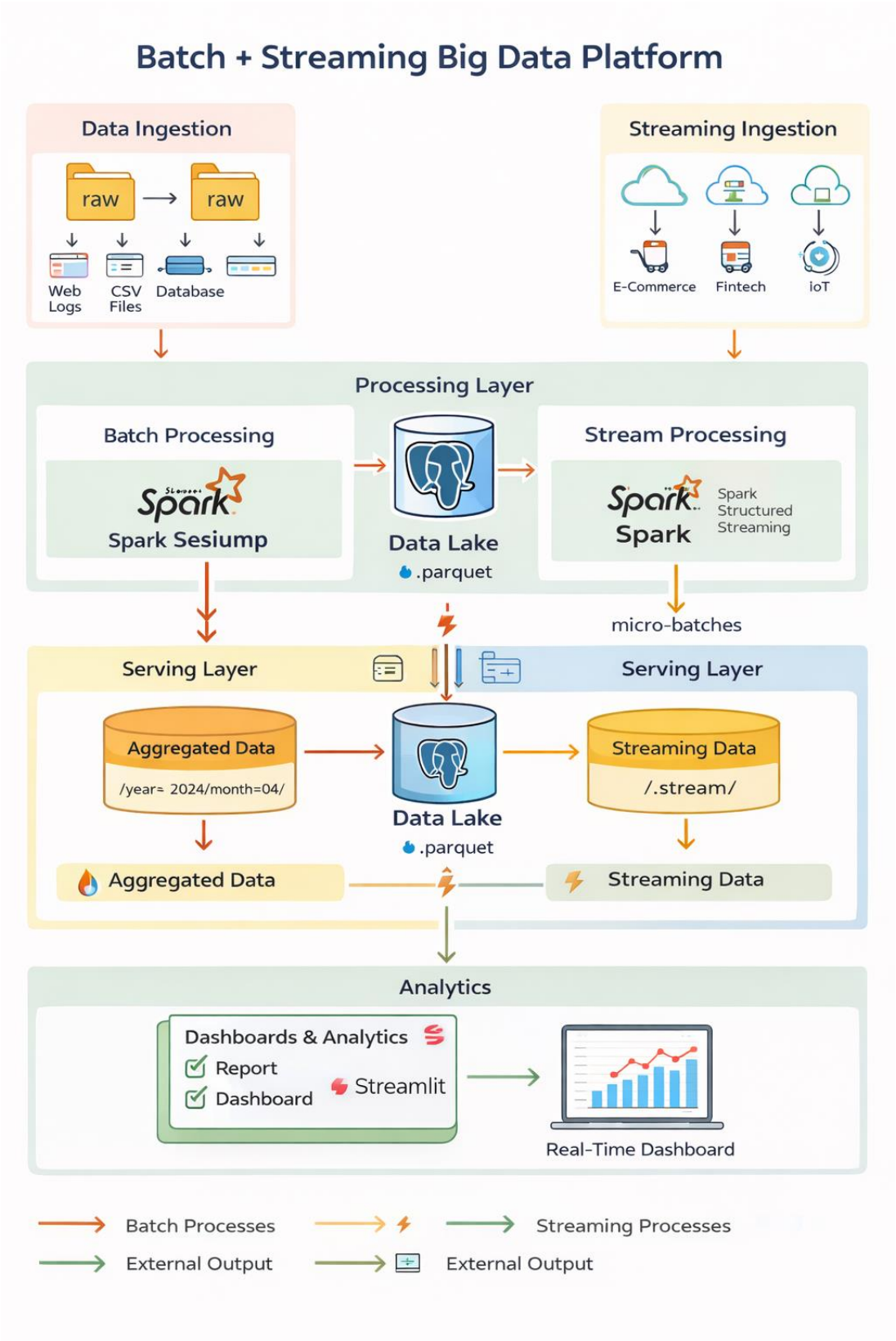
Terminal 3 — Dashboard

```
python -m streamlit run dashboard/dashboard_streamlit.py
```

Gunakan : **Ctrl + C** pada masing-masing terminal.

Jika tidak dihentikan, maka `streaming_processing` akan terus berjalan, `transaction_generator` **akan terus bertambah,** `dashboard_streamlit` akan terus **Update,** dan jangan heran kalau **notebook terasa berat.**

Diagram Arsitektur Praktikum 1–4



Penjelasan ringkas dari Diagram Arsitektur Praktikum 1–4 (*Batch + Streaming Big Data Platform*) di atas.

1 Data Ingestion (Sumber Data)

Tahap pertama adalah **mengumpulkan data dari berbagai sumber**.

Pada diagram terlihat dua jenis ingestion:

1) Batch Data Ingestion

Data dikumpulkan dalam bentuk file atau log, *misalnya:*
- Web Logs
- CSV Files
- Database Export

Data ini biasanya masuk ke folder:

`data/raw/`

dan diproses **secara berkala (batch)**.

2) Streaming Data Ingestion

Data yang masuk **secara real-time sebagai event**.

Contoh sumber:
- transaksi e-commerce
- transaksi fintech
- data sensor IoT

Dalam praktikum, streaming disimulasikan melalui:

`stream_data/`

dengan **transaction generator**.

2 Processing Layer (Layer Pemrosesan)

Layer ini bertanggung jawab untuk **mengolah data mentah menjadi data yang siap dianalisis**.

Terdapat dua jenis pemrosesan.

1) Batch Processing

Batch processing menggunakan:

Apache Spark

Proses yang dilakukan:
- cleaning data
- transformasi
- agregasi

Contoh output:

```
data/curated
data/serving/category_revenue
data/serving/avg_transaction
```

Biasanya disimpan dalam format:

```
.parquet
```

yang efisien untuk Big Data.

2) Stream Processing

Stream processing menggunakan:

Spark Structured Streaming

Data diproses **dalam micro-batches**.

Alur streaming:

```
stream_data/
      ↓
Spark Streaming
      ↓
data/serving/stream/
```

Output disimpan dalam **Parquet Data Lake**.

3 Serving Layer

Serving layer adalah **lapisan data yang siap digunakan oleh aplikasi atau dashboard**. Ada dua jenis data yang disajikan.

1) Aggregated Data (Batch)

Hasil analitik batch seperti:

```
category_revenue
avg_transaction
total_revenue
```

Data biasanya dipartisi seperti:

```
year=2024/month=04
```

2) Streaming Data

Data hasil streaming disimpan di:

```
data/serving/stream
```

Berisi file:

```
part-00000.snappy.parquet
```

Data ini terus bertambah setiap batch streaming.

4 Data Lake

Semua data disimpan dalam **Data Lake berbasis Parquet**.

Keuntungan:
- efisien untuk analitik
- scalable
- bisa digunakan oleh berbagai engine

Data lake ini menjadi **pusat penyimpanan pipeline**.

5 Analytics Layer

Layer terakhir adalah **visualisasi dan analitik**.

Pada praktikum digunakan:

Streamlit Dashboard

Dashboard menampilkan:

- *total transactions*
- *revenue per city*
- *top products*
- *revenue trend*
- *live transactions*

Dashboard dapat diakses melalui:

`http://localhost:8501`

6 Alur Data Keseluruhan

Pipeline lengkap pada diagram:

```
Data Sources
     |
     ▼
Data Ingestion
     |
     ▼
Processing Layer
     |
     ▼
Data Lake
     |
     ▼
Serving Layer
     |
     ▼
Analytics Dashboard
```

7 Hubungan dengan Praktikum 1–4

Praktikum	Fokus
Praktikum 1	***Data ingestion & storage***
Praktikum 2	***Batch processing***
Praktikum 3	***Analytics layer***
Praktikum 4	***Streaming + real-time dashboard***

Sehingga kita memahami ***pipeline Big Data*** **secara** ***end-to-end***.

Insight penting

Arsitektur pada diagram ini sebenarnya adalah ***simplified modern data platform*** yang digunakan dalam sistem data *real-time* di perusahaan teknologi besar seperti Uber dan Netflix.

Menyelesaikan praktikum 1 hingga 4 ini pada dasarnya sudah memahami:

- *data ingestion*
- *batch processing*
- *stream processing*
- *data lake architecture*
- *real-time analytics.*